IOT REPORT

Prepared by Ilamaran Raju Pa14 (S10257054C)

Table of contents

Introduction
Problem Statement
System Requirements
System Architecture
Prototype
Code

Introduction

In Singapore's tropical climate, where ambient temperatures can soar up to 31°C, cultivating temperature-sensitive crops like strawberries presents a significant challenge. These plants thrive in temperatures below 21°C, requiring careful environmental control to ensure optimal growth.

To address this issue, this project focuses on developing a temperature sensor system tailored for farming applications. The system will monitor real-time temperature levels and trigger necessary adjustments—such as activating cooling mechanisms—to

maintain the ideal growing conditions for strawberries and similar crops.

Problem Statement

In urban farming, maintaining optimal growing conditions is essential for maximizing crop yields and ensuring high-quality produce. Real-time and precise temperature monitoring plays a vital role, as different crops thrive within specific temperature ranges. By integrating temperature sensors, farmers can optimize resource usage—such as water and energy—reducing operational costs and minimizing environmental impact.

Additionally, effective temperature regulation helps prevent pest infestations and diseases by eliminating conditions that Favor their growth. Consistently maintaining ideal temperatures and lightings supports year-round production, ensuring a steady supply of fresh produce to meet urban demand. Moreover, when paired with automated climate control systems, temperature sensors enable precise and efficient climate management, enhancing both the sustainability and productivity of urban farming operations.

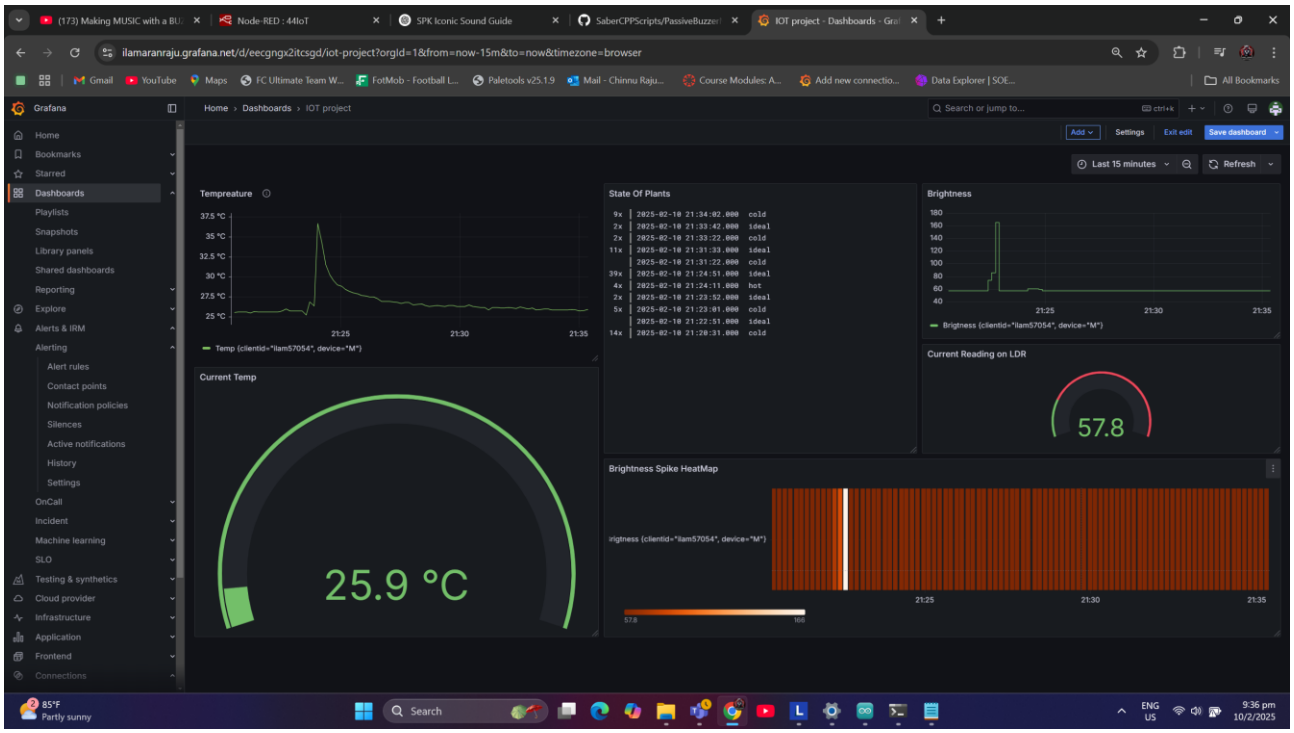
System Requirement

This system enables wireless temperature sensing and provides farmers with real-time data on the temperature conditions inside an urban farm. To ensure optimal plant growth, the system includes visual and auditory alerts for temperature fluctuations.

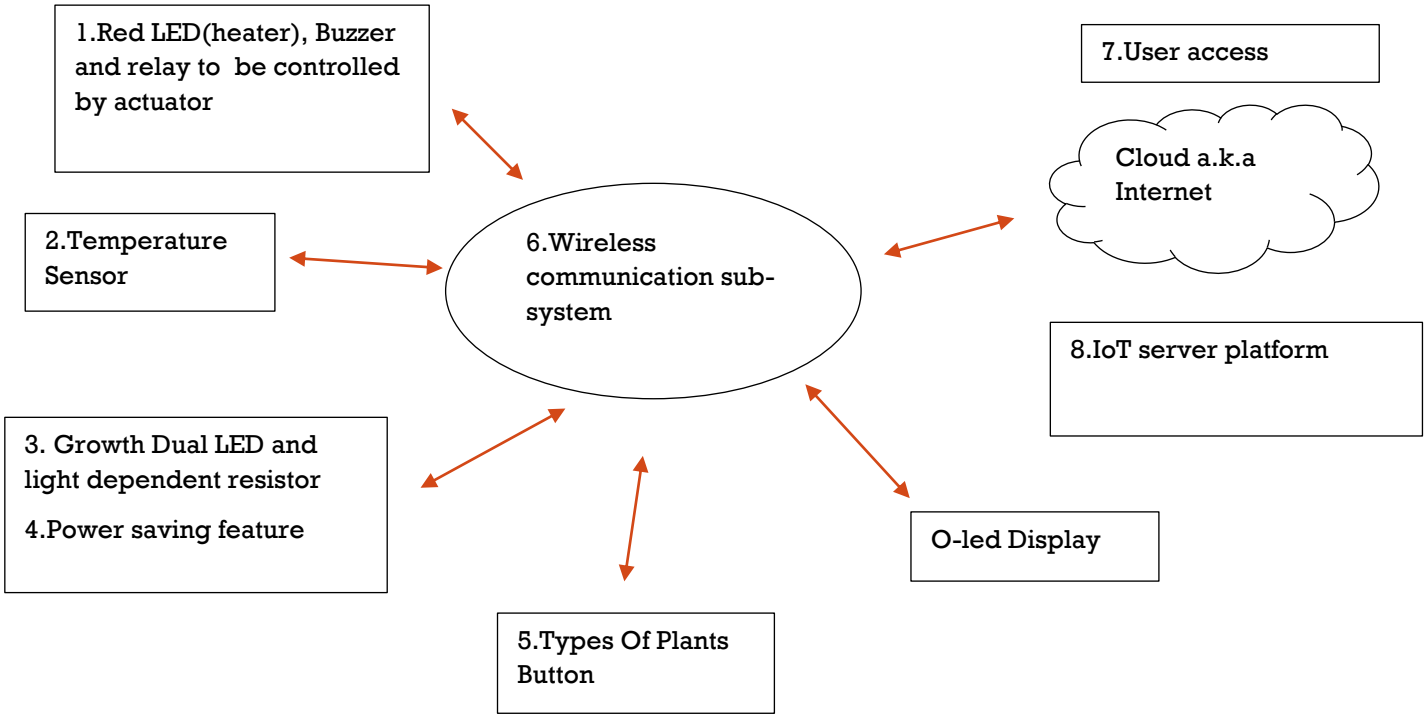
A temperature sensor (see Fig. 1) continuously monitors the ambient temperature. If the temperature exceeds the optimal range, the Relay turns on turning on a cooling fan, and a buzzer sounds to alert farmers. When the temperature remains within the optimal range, nothing happens, indicating stable conditions. When its temperature is below the optimal temp a heater which is indicated by a Red LED is turned on .

Additionally, the system integrates a light-dependent resistor (LDR) and an LED growth light. This feature ensures that plants receive adequate illumination even at night, compensating for the absence of natural sunlight and promoting continuous growth. More over it has a Power Saving Feature .

Figure 1



System Architecture



1. Red LED(heater), Buzzer and relay to be controlled by actuator – When temperature drops below optimal temperature the "heater "LED turn on while the buzzer alerts the farmers by buzzing 2 times to alert the farmers

2. Temperature Sensor - An embedded electronics module featuring the MCP9701A temperature sensor to detect temperature changes and transmit the sensed data every 5 seconds via MQTT over Wi-Fi to a Node-RED IoT server platform.

3. Growth Dual LED and light dependent resistor- The LDR monitors ambient light intensity and automatically activates the LED growth lights when the intensity falls below a predefined threshold.

4. Power saving feature- It is pre-configured that it will turn on a different number of growth lights depending on the darkness

5. O-led Display – able to monitor the temperature actively

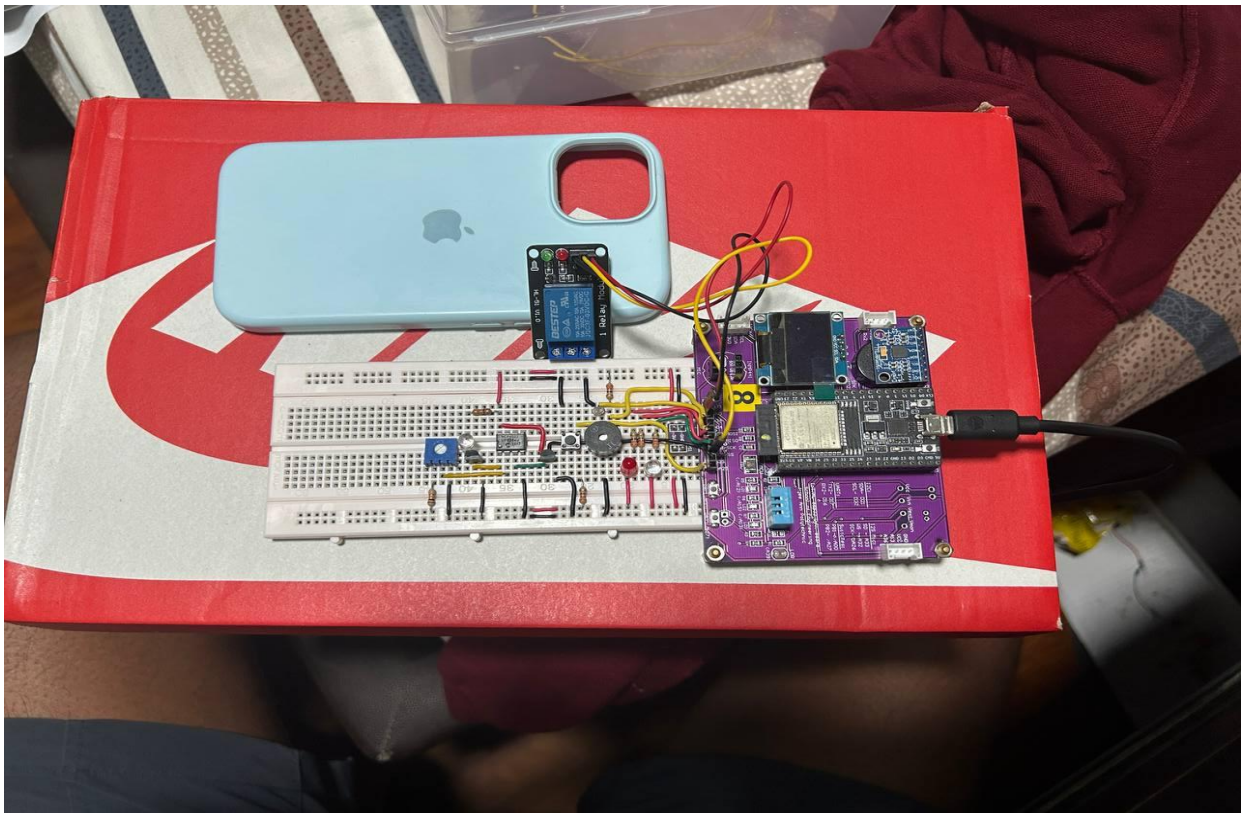
6. Types Of Plants Button- You can change the type of plants you want to grow and set temperature limits on them additionally You can swap modes from plant 1 to plant 2

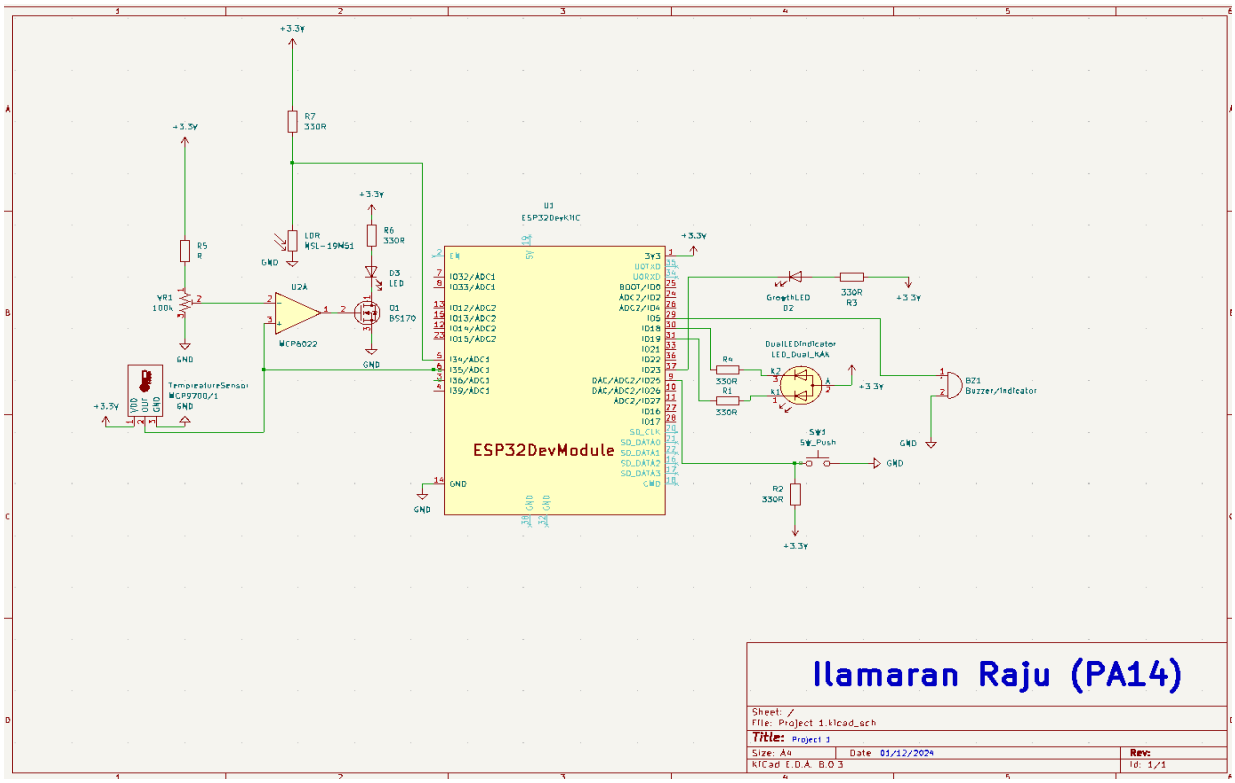
7. Wireless communication sub-system- A Wi-Fi system will be used to connect the light-sensing module to the IoT server platform for internet connectivity.

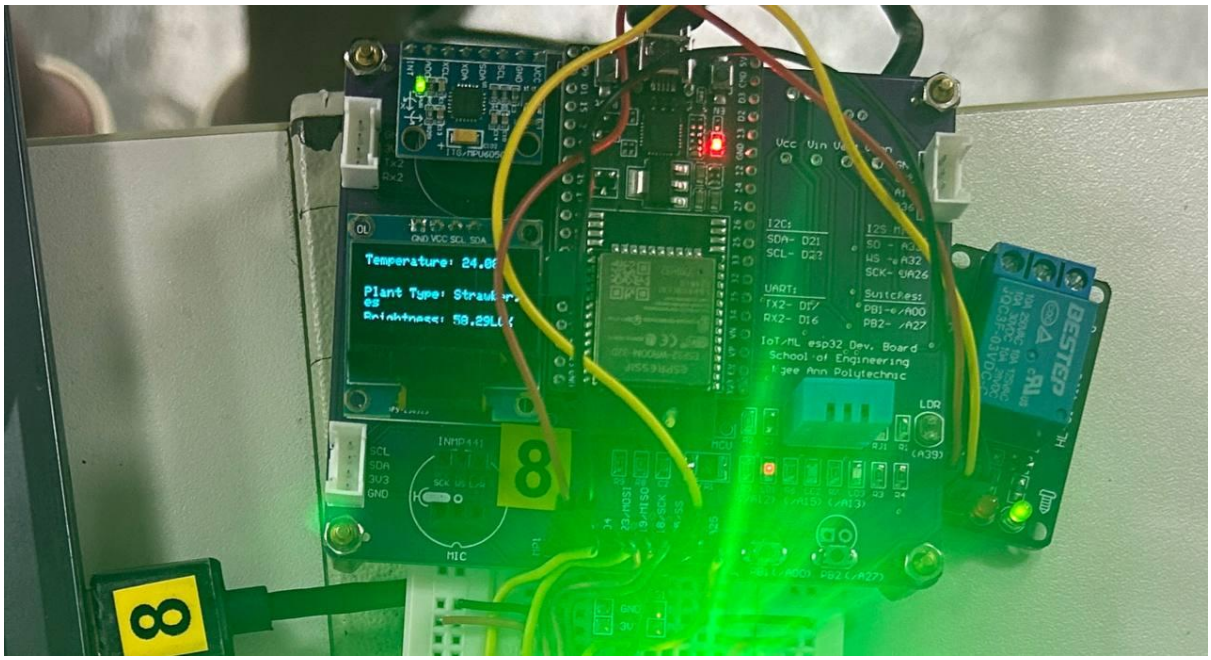
8. User access- a client browser (through PC or handphone) to monitor real time sensor information via a dashboard on Grafana.

9. IoT server platform- Node-RED received the information, processed it, made adjustments, and generated an output with instructions, which were then sent back to the code for execution.

Prototype







Reflection

This project showcases the innovative integration of technology in urban farming, emphasizing the importance of real-time temperature monitoring for optimal plant growth. By utilizing IoT and automation, we not only enhance productivity but also promote sustainability within urban agriculture. The hands-on experience in developing a user-friendly system enriched my understanding of smart farming solutions. This is discussed in detail on pages 5 to 6.

Node-RED plays a pivotal role in the Internet of Things (IoT) ecosystem, providing a user-friendly platform for wiring together devices, APIs, and online services. It facilitates seamless integration and visualization of data streams from sensors, enhancing real-time monitoring. When paired with

Grafana, it transforms raw data into insightful dashboards, empowering users to make informed decisions regarding environmental conditions in urban farming. This synergy enhances efficiency, sustainability, and ultimately aids in maximizing crop yields. By leveraging these technologies, farmers can optimize resource management and respond proactively to fluctuations in growing conditions. The significance of these tools in modern agriculture cannot be overstated, as they promote innovation and resilience within the sector. (Pages 5-6)

Code

```
/* 44IoT Activities development code */

#include <WiFi.h>

#include <WiFiClientSecure.h>

#include <PubSubClient.h>

#include <ArduinoJson.h>

#include "credentials.h"//These lines include necessary libraries for WiFi connectivity, MQTT communication, JSON
handling, and OLED display control.

#include "parameters.h"

#include "profile.h"

#include "cert.h"

#include <Adafruit_GFX.h>                    // OLED lib

#include <Adafruit_SSD1306.h>                // OLED lib


#include <Wire.h>                            // I2C lib


/*** Define Board IO Pins ***/

#define LD1 23

#define DoubleLD1 18          // represents green colour in ithe double led

#define DoubleLD2 19          //represent red colour

#define Buzzer 4

#define LDR2 34  //define board io pins

#define Button 25


#define LD2 15

#define LD3 13                       // Note GPIO13 has multiple use; controls Vmon, Vext & LD3 *

#define LDR 39

#define PB1 0

#define PB2 27

#define A36 36                       // for measuring of Vmon

#define A13 13                       // for controlling Vext and Vmon

#define SPK 2

#define mySensor 35

//OLED Display Configuration

#define SCREEN_WIDTH 128       // OLED display width, in pixels

#define SCREEN_HEIGHT 64       // OLED display height, in pixels
```

```
#define OLED_RESET     -1       // Reset pin
#define SCREEN_ADDRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

#define RAD2DEG  57.3
/*** Define Expansion IO Pins ***/
#define RELAY 5
/*** Global variables ***/
float sensor,ldrsensor;
long timer;
bool currPB2Val, oldPB2Val;                    // States of PB2
char publish_topic[100];
char subscribe_topic[100];
bool button;
String state,plant;
int f;
String line1, line2, line3;

/*** WIFI and MQTT Client setup ***/
#if defined(HIVEMQS)||defined(MOSQUITTOS)     // Read 'profile.h' on #ifdef
 WiFiClientSecure espClient;                   // Creates a client with secure TCP (SSL) connection via WIFI
#else
 WiFiClient espClient;                         // Creates a client with TCP connection via WIFI
#endif
PubSubClient client(espClient);                // Enable the client to connecf with MQTT

void setup() {
  if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
    Serial.println(F("SSD1306 allocation failed"));//Initializes the OLED display and sets up GPIO pins, MQTT topics, and
serial communication.
    for (;;); // Stop the program if OLED initialization fails
}
  /*** GPIO setup ***/
  pinMode(LD1, OUTPUT);
  pinMode(DoubleLD1, OUTPUT);
  pinMode(DoubleLD2, OUTPUT);  //for double led
  pinMode(LDR2, INPUT);
  pinMode(Buzzer, OUTPUT);
  pinMode(Button, INPUT);

  pinMode(LD2, OUTPUT);
  pinMode(LD3, OUTPUT);
  pinMode(A36, INPUT);
```

```
  pinMode(LDR, INPUT);
  pinMode(PB2, INPUT);
  pinMode(mySensor, INPUT);
  pinMode(RELAY, OUTPUT);
  /*** Initialise ***/
  oldPB2Val = digitalRead(PB2);        // PB2 state
  digitalWrite(RELAY, HIGH);           // RELAY pin HIGH to turn off
  digitalWrite(LD1, HIGH);             // all LED off
  digitalWrite(LD2, HIGH);
  digitalWrite(LD3, HIGH);
  digitalWrite(RELAY, HIGH);
  timer= millis();                     // timer
  set_topics();                        // setup MQTT topics

  /*** Setup Serial Communication ***/
  Serial.begin(19200);
  while (!Serial) delay(1);
  /*** Setup WIFI & MQTT Communication ***/
  #if defined(Publish)||defined(Subscribe)
    setup_wifi();
    client.setServer(mqtt_server, mqtt_port);
    client.setCallback(callback);
    client.setKeepAlive(keepalive);
   #ifdef HIVEMQS
    espClient.setCACert(root_ca1);      // Read 'cert.h' on MQTTS; Public Key of HIVEMQ
   #endif
   #ifdef MOSQUITTOS
    espClient.setCACert(root_ca2);      // Read 'cert.h' on MQTTS; Public Key of MOSQUITTO
   #endif
   mqttconnect();
  #endif
}
void Growth()
{
  if (ldrsensor < 59     ){
      Serial.println("  / TOO DARK!!!");

      digitalWrite(DoubleLD1, LOW);
      digitalWrite(DoubleLD2, LOW);

    }
    else if (ldrsensor > 100     ){
      Serial.println("  / No Need for Growth Light");
```

```
      digitalWrite(DoubleLD1, HIGH);
      digitalWrite(DoubleLD2, HIGH);

    }
     else {
      Serial.println("/ Insuffient Lighting");

      digitalWrite(DoubleLD1, LOW);
      digitalWrite(DoubleLD2, HIGH);

     }

}



void Modes()
{
  button = digitalRead(Button);
   if (button == LOW) {
      f++;
      button = true;
      if (f == 2) {
         f = 0;
      }
   } else {
      return;
   }
}

void loop() {
  //Modes();

  if (millis() > timer){                // check timer to start publishing cycle
   /*** read LDR ***/
   Modes();
   sensor = analogRead(mySensor);            // Read Va binary code
   sensor = (sensor +100)*2048/1988;      // 2-pt compensation
   Serial.print("Sense binary:");
   Serial.print((short)sensor);
   sensor = sensor * 0.0008;            // Convert binary to voltage by x3.3/4095 ( = x0.0008)
   Serial.print(" / Sense Voltage Va (V):");
```

```
  Serial.print(sensor);

  ldrsensor = analogRead(LDR2);            // Read Va binary code
  ldrsensor = (ldrsensor +100)*2048/1988;      // 2-pt compensation
  ldrsensor = ldrsensor * 0.0008;
  ldrsensor = 200/ldrsensor;
  Serial.print("/ ldr value:");
  Serial.print(ldrsensor);

  sensor = ((sensor-0.4)/0.0195);
  Serial.print(" Tempreature : ");
   Serial.print(sensor);

   line1 = "Temperature: " + String(sensor) + "C";
  line2 = "Plant Type: " + String(plant);
  line3 = "Brightness: " + String(ldrsensor)+ "LUX";


   digitalWrite(DoubleLD1, HIGH);
     digitalWrite(DoubleLD2, HIGH);
     Growth();



 if (f == 0) {
  Serial.print(" Carrot");
   plant="Carrot";
} else if (f == 1) {
 Serial.print(" Strawberies");
   plant="Strawberies";
}
oleddisplay();  // Update OLED display

#if defined(Publish)
    /*** JSON Object ***/
    DynamicJsonDocument doc(256);                    // JSON Object format: {"key1":value1,"key2":value2,...}
    doc["v"] = (short)(sensor*100);                  // 1st property eg. sense value
    doc["l"] = (short)(ldrsensor*10);
    doc["m"] = (short)(f+1);
    doc["x"] = (state);
    doc["r"] = (short)! digitalRead (RELAY);              // 2nd property eg. voltage of sensor
    char mqtt_message[256];
    serializeJson(doc, mqtt_message);
```

```
    /*** MQTT Publish ***/
    client.publish(publish_topic, mqtt_message, false);
    Serial.println("Published Topic: "+String(publish_topic));
    Serial.println("Published Message: "+String(mqtt_message));
  #endif


  /*** Set timer for next Transmit; non-blocking ***/
  timer = millis() + max(mintxinterval, 10000);  }        //reset timer; end of publishing cycle


   /*** client.loop allow the client to process incoming subscribed messages thru callback() ***/
   #if defined(Publish) || defined(Subscribe)
    if(!client.loop()) mqttconnect() ;
   #endif

   /*** Check PB2 press ***/
   if (pb2PressedChk()){
    if (sensor < 100) {
      digitalWrite(LD1, !digitalRead(LD1));
      }                          ;
   }
}


/******************** Functions **********************/

/*** MQTT Call back Method ***/
void callback(char* topic, byte* payload, unsigned int length) {
 String incommingMessage = "";
 for (int i = 0; i < length; i++) incommingMessage+=(char)payload[i];
 Serial.println("Message arrived ["+String(topic)+"]"+incommingMessage);
 if( String(topic).equalsIgnoreCase(subscribe_topic)){
              //check if incoming topic == subscribe_topic
   if (incommingMessage.equalsIgnoreCase("COOLING FAN ON"))
       { digitalWrite(RELAY,LOW);
       digitalWrite(Buzzer, HIGH);
      delay(1000);
      digitalWrite(Buzzer, LOW);
       digitalWrite(LD1, HIGH);
       Serial.println("  / Its scorching HOT!!!");
```

```
        state="hot";

            }

        else if(incommingMessage.equalsIgnoreCase("HEATER ON"))
        {
digitalWrite(LD1, LOW);
digitalWrite(RELAY,HIGH);
digitalWrite(Buzzer, HIGH);
        delay(500);
        digitalWrite(Buzzer, LOW);
        delay(500);
        digitalWrite(Buzzer, HIGH);
        delay(500);
        digitalWrite(Buzzer, LOW);
        Serial.println("  / Its Too COLD!!!");
        state="cold";
        }
    else
        {
          digitalWrite(RELAY,HIGH);
          digitalWrite(LD1, HIGH);
          Serial.println("/ Idel Tempreature");
          state="ideal";
                        }
  }
}


/*** Connect to WiFi ***/
void setup_wifi() {
  delay(10);
  Serial.print("\nConnecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print("w");
  }
  Serial.println("\nWiFi connected\nIP address: ");
  Serial.println(WiFi.localIP());
}
```

```
/*** Connect to MQTT Broker ***/
void mqttconnect() {
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    if (client.connect(clientid, mqtt_username, mqtt_password)) {
      Serial.println("connected");
       #if defined(Subscribe)
         client.subscribe(subscribe_topic, QoS);       // subscribe topic
       #endif
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");       // Wait 5 seconds before retrying
      delay(5000);

    }
  }
}

/*** Topics setup ***/
void set_topics(){
  sprintf(publish_topic, "%s/%s/%s", appname,clientid,device);
  sprintf(subscribe_topic, "DL/%s/%s/%s", appname,clientid,device);
}

/*** Pushbutton check ***/
 bool pb2PressedChk() {                    // returns true if PB is pressed, false if not pressed
 currPB2Val = digitalRead(PB2);
  if(currPB2Val != oldPB2Val) {            // if there is transition
   oldPB2Val = currPB2Val;
    if(currPB2Val == LOW) {                // if transition is HI to LO
     return true; }
  }
 return false;
 }
void oleddisplay() {
  display.clearDisplay();                 // clear screen
  display.setTextSize(1);                 // set font size
  display.setTextColor(WHITE);            // set color
  display.setCursor(0,0);                 // line1 @row0
  display.println(line1);
  display.setCursor(0,25);                // line2 @row25
```

```
 display.println(line2);
 display.setCursor(0,45);              // line3 @row45
 display.println(line3);
 display.display();                    // print to screen
}
/*** Battery monitor ***/
float batterymonitor() {               // For IoT Power Mgt
 digitalWrite(A13, HIGH);              // High -> Vmon
 delay(1);
 float reading = analogRead(A36);      // ADC read Vmon
 reading = (reading +100)*2048/1988;   // 2-pt compensation
 reading = reading * 0.0008;           // Convert binary to voltage: x3.3/4095 (ie. = x0.0008)
 reading = (2*reading/Vbatmax) * 100;  // ?
 return reading;
}
```